

BMS Predictive Maintenance

Deep Learning Pipeline Report

SOH Estimation Using Deep Learning
Battery A (5s1p, LG 18650HG2 NMC)

5 DL Models x 2 Feature Strategies = 10 Pipelines
LSTM | GRU | 1D-CNN | MLP | Transformer

Birol - BMS Predictive Maintenance Project

Table of Contents

1.	Project Overview & ML vs DL	3
2.	Data & Pipeline Steps	4
3.	Key Difference: Sequences & Scaling	6
4.	DL Models Explained	8
5.	Results & Rankings	12
6.	Key Findings & Analysis	14
7.	ML vs DL Final Comparison	17
8.	Limitations & Caveats	18
9.	Project Structure	19
10.	Next Steps	20

1. Project Overview

Same Problem, Different Approach

This is the same SOH estimation problem as the ML pipeline: predict battery State of Health from operational data (voltage, current, temperature, SOC). The difference is the models used - instead of tree-based algorithms (Random Forest, XGBoost), we use neural networks that can learn temporal patterns from sequences of battery cycles.

ML vs DL: Key Differences

Aspect	ML (1-ML)	DL (2-DL)
Models	RF, XGBoost, LightGBM, etc.	LSTM, GRU, CNN, MLP, Transformer
Trend Features	Manually computed	Not needed (learned from sequence)
Feature Count	~200 (with trends) or 30	~85 (no trends) or 30
Feature Scaling	Not required	Required (StandardScaler)
Input Shape	2D: (samples, features)	3D: (samples, 5, features)
Temporal Context	None (each row independent)	Last 5 cycles together
Training	Single fit (seconds)	Epochs + batches (minutes)
Best Result	ExtraTrees: MAE=0.064%	GRU: MAE=0.249%

2. Data & Pipeline Steps

Data (Same as ML)

Property	Value
Source	recording_senaryo_1_Battery_A_N100.mat
Battery	5s1p LG 18650HG2 NMC
Sampling	1 Hz, ~2.3M rows, ~100 cycles
SOH Range	100% -> 80%

Pipeline Overview (9 Steps)

Step	Description	Same as ML?
1	Raw signal visualization (EDA)	Yes
2	mV -> V conversion	Yes
3	Cycle boundary detection	Yes
4	Feature extraction (~85 features)	Yes
5	Correlation analysis	Yes
6	Feature selection (All or Top 30)	Yes
7	Feature scaling + sequence prep	NEW (DL only)
8	Model training (baseline)	Different model
9	Hyperparameter tuning	Different params

Steps 1-4 are identical to the ML pipeline - same .mat file reading, same cycle detection, same feature extraction. The key difference starts at Step 5: we do NOT compute trend features (rolling mean, delta, rate of change). DL sequence models learn these patterns automatically.

Step 7: Feature Scaling + Sequence Preparation (NEW)

Why Scaling is Required for DL

Tree-based ML models split data at threshold points - the scale doesn't matter. Neural networks multiply inputs by weights and sum them. If voltage is 3.5V and temperature is 30C, the temperature will dominate just because it's a bigger number. StandardScaler normalizes each feature to mean=0, std=1, giving all features equal influence.

Sequence Preparation (Sliding Window)

For sequence models (LSTM, GRU, CNN, Transformer), we create a sliding window of the last 5 cycles for each prediction. Early cycles with insufficient history are zero-padded.

```
Cycle 1: [pad, pad, pad, pad, cycle1] -> SOH_1
Cycle 3: [pad, pad, cycle1, cycle2, c3] -> SOH_3
Cycle 5: [c1, c2, c3, c4, c5] -> SOH_5
Cycle 50: [c46, c47, c48, c49, c50] -> SOH_50
```

```
Shape: (n_samples, 5, n_features) -- 3D tensor
```

MLP is the exception - it uses flat input (no sequences), same as ML models but with a neural network. This serves as the DL baseline.

3. DL Models Explained

Three Families of Neural Networks

Family	Input	Memory	Best For	Our Model
ANN	Fixed vector	None	Tabular data	MLP
CNN	Grid/signal	None	Local patterns	1D-CNN
RNN	Sequence	Yes	Time series	LSTM, GRU
Attention	Sequence	Via attention	Long sequences	Transformer

LSTM (Long Short-Term Memory)

Category: RNN (Recurrent Neural Network)

LSTM processes cycles one by one, maintaining a memory cell with three gates:

- Forget Gate: decides what to discard from memory
- Input Gate: decides what new information to store
- Output Gate: decides what to output

Our architecture: Bidirectional LSTM (reads forward AND backward) with 2 layers. This captures patterns in both directions of the sequence.

Parameter	Baseline	Tuning Range
hidden_dim	64	32, 64, 128
n_layers	2	1, 2, 3
learning_rate	0.001	0.0005, 0.001, 0.005
dropout	0.2	fixed
seq_len	5	fixed

GRU (Gated Recurrent Unit)

Category: RNN (simplified LSTM)

Same concept as LSTM but with only 2 gates (reset + update) instead of 3. This means fewer parameters, faster training, and often similar or better performance on small datasets. Think of GRU as 'LSTM lite'.

Our GRU achieved the best DL result (MAE=0.249%) - its simplicity is an advantage with only 100 training cycles.

1D-CNN (1D Convolutional Neural Network)

Category: CNN

Applies learned filters (kernels) that slide across the cycle sequence. Two Conv1d layers detect local patterns (e.g., 'voltage spread increased for 3 consecutive cycles'), followed by global average pooling to produce a single feature vector.

Unlike LSTM/GRU which process sequentially, CNN processes the entire sequence at once. However, with only 5 timesteps, there isn't much spatial structure for CNN to exploit - which explains its weaker performance.

Parameter	Baseline	Tuning Range
n_filters	64	32, 64, 128
dropout	0.2	0.1, 0.2, 0.3
learning_rate	0.001	0.0005, 0.001, 0.005
kernel_size	3	fixed

MLP (Multi-Layer Perceptron)

Category: ANN (fully connected)

The simplest DL model - no sequence processing. Each cycle is treated independently, just like ML models. Three hidden layers with BatchNorm and Dropout.

Purpose: DL baseline. If sequence models don't beat MLP, temporal information isn't helping.

Key finding: MLP Baseline was catastrophic (MAE=13.58%) because default hyperparameters were terrible. After tuning, MLP Tuned achieved MAE=0.787% - showing that DL is extremely sensitive to hyperparameter choices.

Transformer (Encoder-Only)

Category: Attention-based

The architecture behind GPT, BERT, and ChatGPT - adapted for our SOH prediction task.

Key concept: Self-Attention. Instead of processing cycles sequentially (LSTM) or with local filters (CNN), the Transformer lets each cycle 'attend' to all other cycles simultaneously. It learns which cycles are most relevant for prediction.

Requires Positional Encoding because Transformer has no inherent notion of order - we must explicitly tell it which cycle is 1st, 2nd, etc.

On our small dataset, Transformer performs moderately (MAE=0.441%). Its true power emerges with larger datasets and longer sequences.

Parameter	Baseline	Tuning Range
-----------	----------	--------------

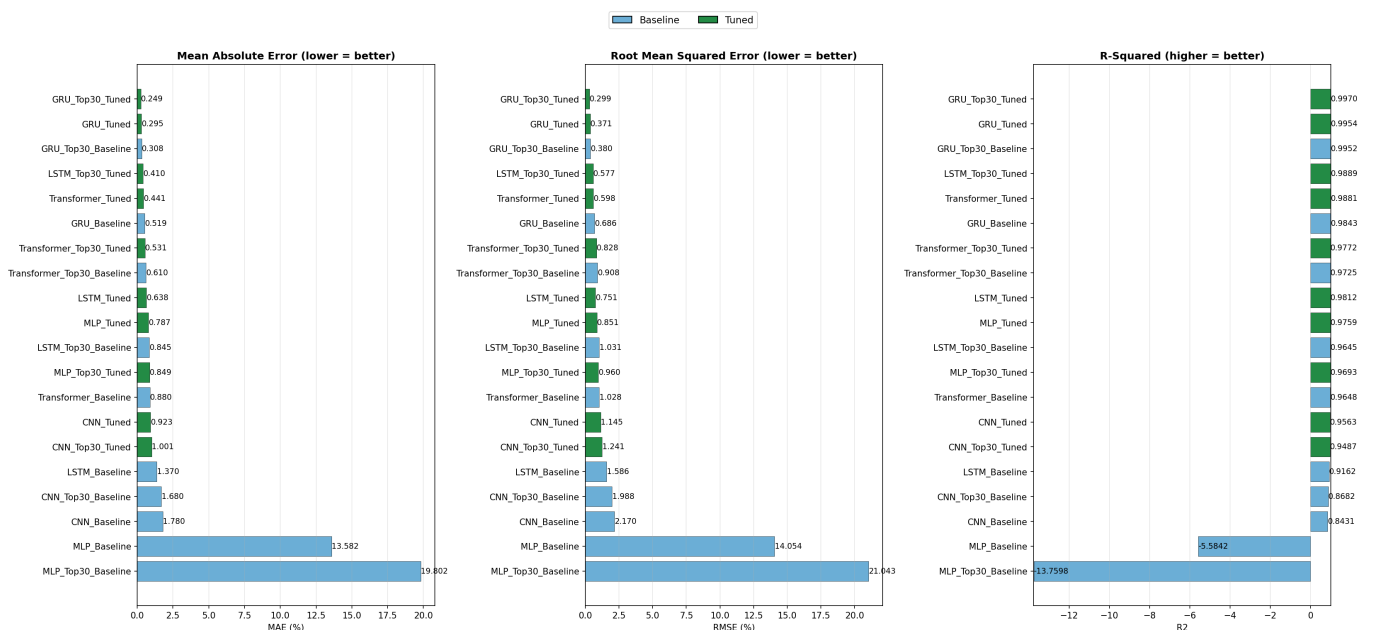
d_model	64	32, 64, 128
nhead	4	fixed
n_layers	2	1, 2, 3
learning_rate	0.001	0.0005, 0.001, 0.005

4. Results & Rankings

All DL Models Ranked by Test MAE

#	Model	MAE	RMSE	R2	CV MAE
1	GRU_Top30_Tuned	0.249%	0.299%	0.9970	0.598%
2	GRU_Tuned	0.295%	0.371%	0.9954	0.802%
3	GRU_Top30_BL	0.308%	0.380%	0.9952	0.795%
4	LSTM_Top30_Tuned	0.410%	0.577%	0.9889	--
5	Transformer_Tuned	0.441%	0.598%	0.9881	1.025%
6	GRU_BL	0.519%	0.687%	0.9843	3.790%
7	TF_Top30_Tuned	0.531%	0.828%	0.9772	0.884%
8	TF_Top30_BL	0.610%	0.908%	0.9725	0.625%
9	LSTM_Tuned	0.638%	0.751%	0.9812	3.350%
10	MLP_Tuned	0.787%	0.851%	0.9759	0.973%
11	LSTM_Top30_BL	0.845%	1.031%	0.9645	--
12	MLP_Top30_Tuned	0.849%	0.960%	0.9693	1.452%
13	TF_BL	0.880%	1.028%	0.9648	0.879%
14	CNN_Tuned	0.923%	1.146%	0.9563	1.719%
15	CNN_Top30_Tuned	1.001%	1.241%	0.9487	3.863%
16	LSTM_BL	1.370%	1.586%	0.9162	1.480%
17	CNN_Top30_BL	1.680%	1.988%	0.8682	7.310%
18	CNN_BL	1.781%	2.170%	0.8431	3.689%
19	MLP_BL	13.58%	14.05%	-5.584	47.42%
20	MLP_Top30_BL	19.80%	21.04%	-13.76	42.98%

DL Model Comparison - SOH Estimation



5. Key Findings & Analysis

Finding 1: GRU is the Best DL Model (MAE = 0.249%)

GRU Top30 Tuned achieved the lowest DL MAE. Its simpler architecture (2 gates vs LSTM's 3) makes it more parameter-efficient, which is crucial with only 100 training cycles. GRU proves that simpler models often win on small datasets.

Finding 2: GRU > LSTM on Small Data

Model	Best MAE	Parameters	Why
GRU Top30	0.249%	Fewer (2 gates)	More efficient
LSTM Top30	0.410%	More (3 gates)	Needs more data

Finding 3: Feature Selection Still Matters in DL

Model	All Features	Top 30	Winner
GRU	0.295%	0.249%	Top 30
LSTM	0.638%	0.410%	Top 30
Transformer	0.441%	0.531%	All
CNN	0.923%	1.001%	All
MLP	0.787%	0.849%	All

RNN models (GRU, LSTM) benefit from Top 30 selection - extra features add noise that degrades sequential learning. CNN, MLP, and Transformer prefer more features. The lesson: 'DL learns everything automatically' is NOT true with 100 samples.

Finding 4: Hyperparameter Tuning is Critical

Model	Baseline MAE	Tuned MAE	Improvement
MLP	13.582%	0.787%	17x better!
LSTM	1.370%	0.638%	2.1x
GRU	0.519%	0.295%	1.8x
CNN	1.781%	0.923%	1.9x
Transformer	0.880%	0.441%	2.0x

DL is far more sensitive to hyperparameters than ML. MLP Baseline had catastrophic $R^2 = -5.58$ (worse than random). After tuning: $R^2 = 0.976$. In ML, baseline RF already gave $R^2 = 0.999$.

Finding 5: CNN Underperforms on Short Sequences

CNN (MAE ~0.92-1.78%) consistently underperforms RNNs and Transformer. 1D convolution filters are designed for local pattern detection in long sequences. With only 5 timesteps, there isn't enough spatial structure. CNN would likely improve with longer sequences (seq_len=20+).

Finding 6: Transformer - Moderate on Small Data

Transformer Tuned achieved MAE=0.441%, ranking 5th. Self-attention is powerful but parameter-heavy. With 100 cycles and seq_len=5, there isn't enough data to leverage Transformer's full potential. On larger datasets (1000+ cycles), Transformer typically outperforms RNNs.

Finding 7: MLP Baseline is a Cautionary Tale

MLP Baseline: MAE=13.58%, R2=-5.58. This means the model's predictions were WORSE than simply predicting the mean SOH for every cycle. The random weight initialization, combined with suboptimal learning rate and architecture, led to complete failure.

Lesson: DL models are NOT plug-and-play. They require careful hyperparameter selection. ML models (especially tree-based) are much more forgiving with default parameters.

6. ML vs DL Final Comparison

Best ML		Best DL
Model	ExtraTrees Top30 BL	GRU Top30 Tuned
MAE	0.064%	0.249%
RMSE	0.079%	0.299%
R2	0.9998	0.9970
Winner	ML	--

ML wins on this dataset. This is expected and well-documented in the literature:

1. Tree-based models excel on small tabular data (<1000 samples)
2. DL needs more data to learn complex patterns
3. ML had manually computed trend features (~200 features) - domain knowledge advantage
4. DL models are more sensitive to hyperparameters

However, DL has unique advantages that don't show on this dataset:

- Extrapolation: DL can potentially predict beyond training range (trees cannot)
- Transfer learning: Pre-train on one battery, fine-tune on another
- Scalability: DL improves with more data, ML plateaus

When Would DL Win?

Data Size	Expected Winner	Why
<500 cycles	ML	Not enough data for DL
500-5000	Close / DL starts winning	DL patterns emerge
5000+	DL	DL fully leverages patterns
Transfer learning	DL	Pre-trained models adapt fast

7. Limitations & Caveats

100 Cycles is Very Limited for DL

Deep learning typically needs thousands of samples. With 100 cycles, all DL models are data-starved. The results would likely be very different with 10 batteries x 100 cycles = 1000 training samples.

Feature Engineering Still Critical

We still manually designed domain features (voltage spread, proxy IR, etc.) in Step 4. True 'end-to-end DL' would take raw 1 Hz signals directly - but with 100 cycles of 23000-second signals, this is computationally infeasible and would overfit massively.

Interleaved MAE is Optimistic

Same caveat as ML. GRU's interleaved MAE=0.249% but CV MAE=0.598%. Real-world performance is closer to the CV MAE. Fold 5 (chronological test) would be even higher.

Non-Deterministic Training

DL training involves random weight initialization and stochastic gradient descent. Results vary slightly between runs. We use random seeds for reproducibility, but GPU operations can introduce minor variations.

8. Project Structure

```

2-DL/
|-- LSTM/
|   |-- LSTM.py, LSTM_Top30.py
|   |-- Output_LSTM/, Output_LSTM_Top30/
|-- GRU/
|   |-- GRU.py, GRU_Top30.py
|-- CNN/
|   |-- CNN.py, CNN_Top30.py
|-- MLP/
|   |-- MLP.py, MLP_Top30.py
|-- Transformer/
|   |-- Transformer.py, Transformer_Top30.py
|-- DL_Comparison.py
|-- DL_Comparison.csv / .png
|-- DL_Pipeline_Report.pdf (this document)
|-- README.md

```

Output Files Per Model

File	Content
step[1-7]_*.png	Pipeline step visualizations
step8_*.png	Training: metrics, prediction, residuals, loss curve
step9_*.png	Tuning: baseline vs tuned, tuned prediction
model_*_baseline.pt	Baseline PyTorch model weights
model_*_tuned.pt	Tuned PyTorch model weights
scaler.joblib	StandardScaler for inference
feature_columns.txt	Feature list for inference
Result_*.csv	Summary with overfitting checks + CV MAE

9. Next Steps

Battery B Inference

Apply trained DL models to Battery B (unseen battery, ground truth SOH=88%). Same feature extraction pipeline, load saved scaler and model weights, predict SOH.

ML + DL Combined Comparison

Create a unified comparison chart showing all ML (12 models) and DL (20 models) side by side. This gives the complete picture of model performance on this dataset.

Transfer Learning

Pre-train DL models on Battery A, then fine-tune on Battery B with minimal data. This is where DL is expected to significantly outperform ML - leveraging learned degradation patterns from one battery to predict another.

Larger Datasets

Combine multiple batteries (A + B + additional simulations) to create a dataset of 500+ cycles. This would allow DL models to fully leverage their capacity and likely surpass ML performance.

Ensemble Methods

Combine the best ML model (ExtraTrees, MAE=0.064%) with the best DL model (GRU, MAE=0.249%) through prediction averaging or stacking. Ensemble methods often outperform any single model.